AArch64 Memory & Cache System — Technical Reference Workbook (based on arm64 Cortex A72) 1
Step 0 — Foundational Cache Terminology and Structures 1
0.1 Scope 1
0.2 Cache Line (Example is at the end to visualize the cache structure in memory) 1
0.3 Set and Index 1
0.4 Way and Associativity 2
0.5 Tag 2
0.6 Dirty and Clean Lines 2
Dirty Line 2
Clean Line 2
0.7 WriteBack vs WriteThrough 3
WriteBack 3
WriteThrough 3
0.8 Replacement Policy 3
0.9 Inclusive vs Exclusive 3
Inclusive 4
Exclusive 4
Noninclusive, nonexclusive (NINE) 4
0.10 Snoop Behavior 4
0.11 Summary Table 4
0.12 Exercises (Technical Reference) 5
Exercise 0.1 — Terminology Classification 5
Exercise 0.2 — Associativity Reasoning 5
Exercise 0.3 — Dirty Line Propagation 5
Simplified Example how data cache gets stored in ARM (Developer does not need to program anything for it. This understanding is needed if you want to manage cache using set associative way which we rarely do. Also this will help to understand Cache Thrashing problem later in topic 11.0): 5

A simplified four-way set associative 32KB L1 cache (such as the data cache of the Cortex-A57 processor), with a 16-word (64 byte) cache line length as per below diagram used from arm reference document 6
1. Start from a 64-bit physical address 6
2. Split the address into Tag, Set, Word, and Byte (Key Concept) 6
3. Understand the cache line structure 7
4. Use the Set (Index) bits to choose a set 7
5. Look at all ways inside the selected set 7
6. Compare the Tag field with each way's stored tag 7
7. Use Word and Byte bits to pick the exact byte 8
8. Handle reads, writes, and dirty lines 8

AArch64 Memory & Cache System — Technical Reference Workbook (based on arm64 Cortex A72)

Step 0 — Foundational Cache Terminology and Structures

0.1 Scope

This step defines the fundamental terminology required to interpret cache related architectural documentation. These definitions apply to Cortex-A72/A73 and most ARMv8-A cores.

0.2 Cache Line (Example is at the end to visualize the cache structure in memory)

A **cache line** is the smallest unit of allocation and transfer in the cache hierarchy.

- Cortex-A72/A73 line size: **64 bytes**
- All DC/IC operations operate on cache lines
- All fills, evictions, writebacks, and invalidations occur at line granularity

0.3 Set and Index

Caches are organized into **sets**.

- Each set contains multiple **ways**
- A memory address maps to exactly one set
- The **index** bits of the address select the set

Example:

```
Address → [Tag | Index | Offset]
                    ↑
                selects set
```

0.4 Way and Associativity

A **way** is one entry in a set.

If a cache has *N ways*, it is **N-way set associative**.

Example:

- 4-way set associative cache
- Each set contains 4 possible line slots
- A line can be placed in any of the 4 ways

Associativity reduces conflict misses.

0.5 Tag

The **tag** identifies which memory block is stored in a given cache line.

- Tag + Index uniquely identify a line
- Tag RAM stores tags

- Data RAM stores actual line data

0.6 Dirty and Clean Lines

Dirty Line

- Modified in cache
- Not yet written back to PoC
- Exists only in write-back caches
- Requires DC CVAC / DC CVAU / DC CIVAC to propagate

Clean Line

- Matches PoC
- No writeback required
- Can be evicted without writeback

0.7 Write-Back vs Write-Through

Write-Back

- Writes update cache only
- DRAM updated later (writeback)
- Dirty lines exist
- Cortex-A72/A73 L1D and L2 are write-back

Write-Through

- Writes update both cache and DRAM
- No dirty lines
- Clean operations have no effect
- Rare in ARMv8-A except for special regions

0.8 Replacement Policy

Determines which line is evicted when a set is full.

Common policies:

- Pseudo-LRU (typical for Cortex-A72/A73)
- Random (rare)
- True LRU (expensive)

Replacement policy is microarchitectural and not software-visible.

0.9 Inclusive vs Exclusive

Inclusive

- L2 contains all lines present in L1
- Evicting L2 forces eviction of L1
- Cortex-A72/A73 L2 is **not strictly inclusive**

Exclusive

- L1 and L2 contain disjoint sets of lines
- Rare in ARMv8-A

Non-inclusive, non-exclusive (NINE)

- L2 may or may not contain L1 lines
- Cortex-A72/A73 use this model

0.10 Snoop Behavior

Snoops are coherence messages exchanged between caches.

- L2 snoops L1D for dirty lines
- L1I is not snooped (instructions are refetched from PoU)
- Snoops ensure correctness for DC CVAC / CIVAC operations

0.11 Summary Table

Term	Meaning	Cortex-A72/A73 Behavior
Cache line	Smallest unit (64B)	64 bytes

Set	Group of ways	Indexed by address
Way	Slot in a set	16-way L1D, 8-way L2 (typical)
Tag	Identifies memory block	Stored in tag RAM
Dirty line	Modified, not written back	Yes
Clean line	Matches PoC	Yes
Write-back	Write cached first	Yes
Write-through	Write cached + DRAM	No
Replacement	Eviction policy	Pseudo-LRU
Inclusive	L2 contains L1	No
Exclusive	L1 separate from L2	No
NINE	Non-inclusive, non-exclusive	Yes

0.12 Exercises (Technical Reference)

Exercise 0.1 — Terminology Classification

Classify each item as: *line*, *set*, *way*, *tag*, *dirty*, *clean*.

1. A 64-byte block transferred during DC CVAC
2. The group selected by index bits
3. The slot inside a set
4. The identifier stored in tag RAM
5. A modified line not yet written back
6. A line matching PoC

Exercise 0.2 — Associativity Reasoning

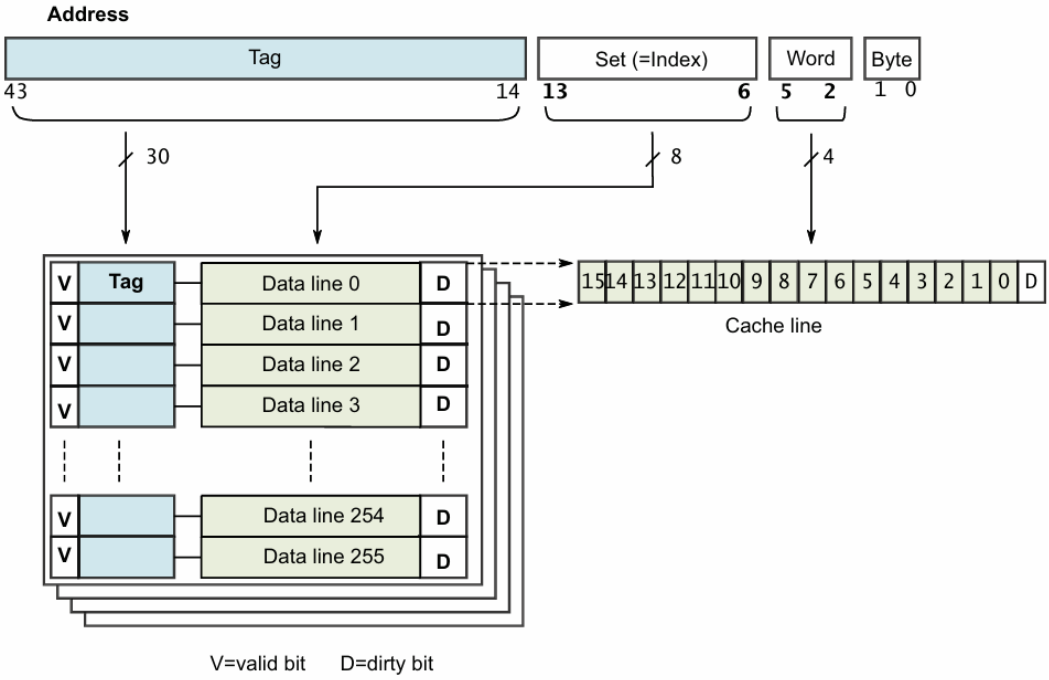
Explain why increasing associativity reduces conflict misses.

Exercise 0.3 — Dirty Line Propagation

Describe the sequence of events when a dirty line in L1D must be made visible to DMA.

Simplified Example how data cache gets stored in ARM (Developer does not need to program anything for it. This understanding is needed if you want to manage cache using set associative way which we rarely do. Also this will help to understand Cache Thrashing problem later in topic 11.0):

A simplified four-way set associative 32KB L1 cache (such as the data cache of the Cortex-A57 processor), with a 16-word (64 byte) cache line length as per below diagram used from arm reference document



1. Start from a 64-bit physical address

The CPU issues a 64-bit physical address for a load or store.

- Think of an address like: `0x0000_1234_5678_ABCD`
- This address points to a single byte in memory
- The cache will decide if that byte is already stored locally

2. Split the address into Tag, Set, Word, and Byte (Key Concept)

The address is divided into fields that each serve a specific role in cache lookup.

| Tag (bits 43–14) | Set/Index (bits 13–6) | Word (bits 5–2) | Byte (bits 1–0) |

- **Tag (43–14):** identifies which memory block is cached
- **Set (13–6):** selects one of 256 sets
- **Word (5–2):** selects one of 16 words in the line
- **Byte (1–0):** selects one of 4 bytes inside that word

3. Understand the cache line structure

Each cache line holds 64 bytes, organized as 16 words of 4 bytes each.

- **1 word = 4 bytes (32 bits)**
- **16 words × 4 bytes = 64 bytes per line**
- The Word+Byte bits (5–0) can address any of those 64 bytes

4. Use the Set (Index) bits to choose a set

The index field selects which group of cache lines to look in.

- Bits **13–6** form an 8-bit index
- **2^8 = 256 sets**, numbered 0–255
- The diagram's "Data line 0...255" corresponds to these 256 sets

5. Look at all ways inside the selected set

Each set contains four possible slots (ways) where the line might reside.

- This is a **4-way set-associative** cache
- For the chosen set, there are **Way 0, Way 1, Way 2, Way 3**
- Each way stores: **Tag + Valid bit + Dirty bit + 64-byte data line**

6. Compare the Tag field with each way's stored tag

Hit or Miss

The tag bits from the address are checked against the tags in all four ways of the selected set.

- Hardware reads all 4 tags in that set in parallel
- If one tag **matches** and its **Valid bit = 1** → potential **cache hit**
- If **no tag matches** → **cache miss** and data must be fetched from lower levels

7. Use Word and Byte bits to pick the exact byte

Once the correct cache line is found, the offset selects the specific location inside it.

- **Word bits (5–2):** choose which of the 16 words (0–15)
- **Byte bits (1–0):** choose which of the 4 bytes inside that word
- Together, they pinpoint the exact byte the CPU requested

8. Handle reads, writes, and dirty lines

Behavior

The cache responds differently depending on whether the access is a read or a write.

- **Read hit:** data is returned directly from the chosen way's 64-byte line
- **Write hit:** data is updated in that line and the **Dirty bit** is set
- **Miss:** a new 64-byte line is fetched; a replacement way is chosen (e.g., pseudo-LRU)
- If the evicted line's Dirty bit = 1, its 64 bytes are written back to the next level

In short:

- The **Set (index)** chooses *which group* of lines to search.
- The **Tag** decides *whether one of those lines is the right memory block*.
- The **Word+Byte** bits pick *the exact byte* inside the 64-byte cache line.

Cortex-A72/A73 is 16-way L1D instead of 4 way as described in above example